

Understanding Git And GitHub

Preksha Samajpati
Department of Computer Science and Engineering
Manipal Institute of Technology, MAHE
Manipal, India
preksha.mitmpl2026@learner.manipal.edu

Abstract— This report documents the completion of Task Two, covering the internal mechanics of Git (commits, branching, merging, rebasing, logs and remote repositories) as practiced through [learnxgitbranching.js.org](https://learnx.in/javascript/learngitbranching.js.org), and these concepts were then used on a demo repository containing the solution files of the problems.

Keywords— Git, version control, GitHub, branching, merging, rebasing, remote repositories.

I. INTRODUCTION

Every large software project eventually asks the same question: how do dozens of people edit the same code without erasing each other's work? Git answers this by treating a project's history not as a single line, but as a tree of snapshots — every save is a checkpoint you can branch from, merge back into, rewind to, or recover even after it seems lost. Git is a **distributed version control system** — software that tracks changes to files over time, so you can see history, revert mistakes, and work on multiple versions of a project in parallel (branches) without them interfering with each other. Every developer has a **full copy of the entire project history** on their own machine, not just the latest snapshot. This means you can commit, branch, and view history completely offline, and there's no single point of failure — if the central server disappears, any clone can restore everything. Git provides features such as clone, commit, push, pull, branching, merging and rebasing for managing project history

Sections II to VIII explain concepts like what a commit is and how commits are stored, what a branch is, the difference between merging and rebasing, how push and pull work with a remote repository, and how merge conflicts and stashing work. Section IX shows this in practice on the local repository made for this task, where the Task 1 Python solutions were committed and pushed.

II. GIT COMMITS

A. GIT Commits:

A commit in a git repository records a snapshot of all the (tracked) files in your directory. It's like a giant copy and paste, but even better.

Git wants to keep commits as lightweight as possible though, so it doesn't just blindly copy the entire directory every time you commit. It records changes to your repository and allows you to track project history, collaborate, or revert to previous states if something breaks.

B. HEAD AND Relative References:

HEAD is the symbolic name for the currently checked out commit -- it's essentially what commit you're working on top of. HEAD always points to the most recent commit which is reflected in the working tree. Most git commands which make changes to the working tree will start by changing HEAD.

Git also allows previous commits to be referenced using relative references such as HEAD^ and HEAD~1 for moving back through commits. These relative references are useful as you can then reference previous commits without using the full commit hash (which looks something like: fed2da64c0efc5293610bdd892f82a58e8cbc5d8)

III. GIT BRANCHING AND BRANCH COMMANDS

A. GIT Branching:

Git branching is a core feature in the Git Version Control System that allows you to diverge from the main line of development to work on new features, bug fixes, or experiments independently. In technical terms, a Git branch is simply a lightweight, movable pointer to a specific commit. By default, Git initializes projects with a primary branch, usually named main or master. Creating a new branch creates a parallel workspace where you can record new commits without affecting the production-ready code on your main branch.

B. Branch Commands

1. List Branches: git branch-Shows all local branches in your repository and highlights your active branch with an asterisk (*).
2. Create a branch: git branch <branch-name>- Creates a new pointer at your current commit but does not switch your active workspace to it.
3. Switch branches: git switch <branch-name>- Moves your active workspace (the HEAD pointer) to the specified branch. (Note: You can also use the older command git checkout <branch-name>).

4. Create and switch instantly: `git switch -c <branch-name>`-A shortcut that creates a brand-new branch and switches you to it immediately. (Equivalent to `git checkout -b <branch-name>`).

IV. GIT MERGING AND REBASING

A. *Git Merge*:

Merging is a way to combine the changes from one branch into another. When you merge a branch, Git creates a new commit called a merge commit that has two parent commits: one from the current branch and one from the branch being merged. Merging preserves the complete history of changes, making it easy to trace back through the commit history. The merge commit provides a clear indication that branches have been combined, offering context for future reference.

B. *Git Rebase*:

Rebasing is a process of moving or combining a sequence of commits to a new base commit. It allows you to take the changes from one branch and reapply them on top of another branch. Git rebase actually rebases the feature branch and merges it with the main branch. In simple words, it moves the entire feature branch to the tip of the main branch. Rebasing creates a linear history, which can make the commit history easier to read and understand.

V. GIT REVERT AND RESTORE, GIT TAG

A. *Git Revert*:

The `git revert` command is used to undo changes introduced by a specific previous commit by creating a completely new commit that applies the exact inverse changes. Unlike destructive commands like `git reset`, `git revert` does not alter or erase your existing commit history. This makes it the safest option for undoing mistakes on public or shared remote branches.

B. *Git Restore*:

Everybody makes a little mess sometimes. You stage a file you didn't mean to, or start an experiment you'd rather throw away. `git restore` is the modern, purpose-built undo button for your working directory and staging area. It comes in two types:

`git restore --staged <file>`: unstage a file (moves it back out of the staging area, keeping edits).

`git restore <file>`: discard your edits to a file entirely.

C. *Git Tag*:

Git tags permanently mark certain commits as "milestones" that you can then reference like a branch. More importantly though, they never move as more

commits are created. You can't "check out" a tag and then complete work on that tag -- tags exist as anchors in the commit tree that designate certain spots.

VI. REMOTE REPOSITORIES: PUSH, PULL AND FETCH

Remote repository is a version of your project that is hosted on the internet, a network, or an external server rather than on your local computer. It serves as a centralized hub or backup that enables multiple developers to collaborate, share changes, and keep their code synchronized.

A. *Git Fetch vs Git Pull*:

The `Git Fetch` command is used to fetch all changes from the remote repository to the local repository. It doesn't make any changes to the current working directory. It stores all the changes in a separate branch called the remote-tracking branch. `git merge` or `git rebase` command is used to merge these changes into our current working directory.

`Git Pull` command is used to fetch all changes from the remote repository to the current working directory. It automatically tries to merge or rebase them into our current working directory. It is the combination of `git fetch` and `git merge` or `git rebase`. It can generate merge conflicts if there are conflict changes between our local and remote branches.

B. *Force Pushing*:

`git push` updates a remote repository with your local commits, but it only works if your local history is perfectly up to date with the remote history.

`git push --force` (or `-f`) bypasses this safety check, completely overwriting the remote repository's history with whatever is on your local machine.

VII. RESOLVING MERGE CONFLICTS

A merge conflict occurs when Git tries to combine two branches that have changed the same lines of a file (or one branch edits a file the other deletes) and cannot automatically determine which version to keep.

When this happens, running `git merge feature-branch` pauses mid-merge and marks the affected files as unmerged, which `git status` will list under "Unmerged paths." Opening the conflicted file reveals Git's conflict markers — <<<<<<< HEAD marking the start of your current branch's version, ===== separating it from the incoming branch's version, and >>>>>> feature-branch marking the end — and resolving the conflict means manually editing the file to keep one version, merge both, or write new code, then deleting all three marker lines.

Once the file reflects the intended result, git add <filename> marks it as resolved, and git commit finalizes the merge with an auto-generated (editable) merge commit message.

VIII. GIT STASHING

Git Stash temporarily shelves (saves) changes you've made to your working copy so you can work on something else, and then come back and re-apply them later. It is incredibly useful when you need to quickly switch branches or pull upstream changes but aren't ready to commit your unfinished work.

IX. DEMONSTRATION

The task began with completing both the Main and Remote tabs on learngitbranching.js.org to build a conceptual foundation in commits, branching, merging, rebasing, and remote operations like fetch, pull, and push, with a screenshot taken of each tab's completion screen.

For the practical demonstration, a local repository was set up inside ~/Documents/TP_2, where all eight Python program files were staged and committed together using git add . followed by git commit -m "Initial commit: Task phase 1 Programs", creating the root commit da4b6db. Running git push -u origin master succeeded, uploading the commit and setting up tracking between the local master branch and origin/master.

```

PrekshaDESKTOP-H33ATTI MINGW64 ~/Documents/TP_2 (master)
$ git add .
PrekshaDESKTOP-H33ATTI MINGW64 ~/Documents/TP_2 (master)
$ git commit -m "Initial commit: Taskphase 1 Programs"
[master (root-commit) da4b6db] Initial commit: Taskphase 1 Programs
8 files changed, 291 insertions(+)
 create mode 100644 tp1.1.py
 create mode 100644 tp1.2.py
 create mode 100644 tp1.3.py
 create mode 100644 tp1.4.py
 create mode 100644 tp1.5.py
 create mode 100644 tp1.6.py
 create mode 100644 tp1.7.py
 create mode 100644 tp1.8.py

PrekshaDESKTOP-H33ATTI MINGW64 ~/Documents/TP_2 (master)
$ git push -u origin main
error: src refspec main does not match any
error: failed to push some refs to 'github.com:PrekshaSamaipati/gitdemo.git'

PrekshaDESKTOP-H33ATTI MINGW64 ~/Documents/TP_2 (master)
$ git push origin --master
Enumerating objects: 10, done.
Counting objects: 100% (10/10), done.
Delta compression using up to 8 threads
Compressing objects: 100% (10/10), 2.95 KiB | 378.00 KiB/s, done.
Writing objects: 100% (10/10), 2.95 KiB | 378.00 KiB/s, done.
Total 10 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
Remote: Resolving deltas: 100% (2/2), done.
To github.com:PrekshaSamaipati/gitdemo.git
 * [new branch] master -> master
branch 'master' set up to track 'origin/master'.

PrekshaDESKTOP-H33ATTI MINGW64 ~/Documents/TP_2 (master)
$ git log --online
da4b6db (HEAD --> master, origin/master) Initial commit: Taskphase 1 Programs

```

Fig. 1 output of git log --online confirming the initial commit.

Next, git pull was run to bring in changes made directly on GitHub — four CSV files (blue.csv, centreline.csv, cones.csv, yellow.csv) that had been added remotely. Since the local branch had no

commits of its own beyond the shared history, Git performed a fast-forward merge, updating the local branch to commit c5b11d1 without needing a separate merge commit.

```

PrekshaDESKTOP-H33ATTI MINGW64 ~/Documents/TP_2 (master)
$ git pull
remote: Enumerating objects: 7, done.
remote: Counting objects: 100% (7/7), done.
remote: Compressing objects: 100% (4/4), done.
remote: Total 6 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
Unpacking objects: 100% (6/6), 1.22 KiB | 43.00 KiB/s, done.
From github.com:PrekshaSamaipati/gitdemo
   da4b6db..c5b11d1 master -> origin/master
Updating da4b6db..c5b11d1
Fast-forward
 blue.csv      | 3 +++
 centreline.csv | 3 +++
 cones.csv     | 5 +++++
 yellow.csv    | 3 +++
 4 files changed, 14 insertions(+)
 create mode 100644 blue.csv
 create mode 100644 centreline.csv
 create mode 100644 cones.csv
 create mode 100644 yellow.csv

PrekshaDESKTOP-H33ATTI MINGW64 ~/Documents/TP_2 (master)
$ git log --online
c5b11d1 (HEAD --> master, origin/master, origin/HEAD) Added the csv files as well
da4b6db Initial commit: Taskphase 1 Programs

```

Fig 2: Git Pull performed as well as a fast-forward merge

To demonstrate branching and conflict resolution, a new branch called conflict was created from the current master. The same line in tp1.5.py was then edited differently on each branch — once while on conflict (committed as "Updated tp1.5 on conflict-branch") and once while back on master (committed as "Updated tp1.5 on master branch") — ensuring both branches diverged from the same shared ancestor with conflicting changes.

Running git merge conflict from master triggered a genuine merge conflict, with Git reporting CONFLICT (content): Merge conflict in tp1.5.py and inserting <<<<<<< HEAD, =====, and >>>>>>> conflict markers into the file to delimit the two competing versions.

```

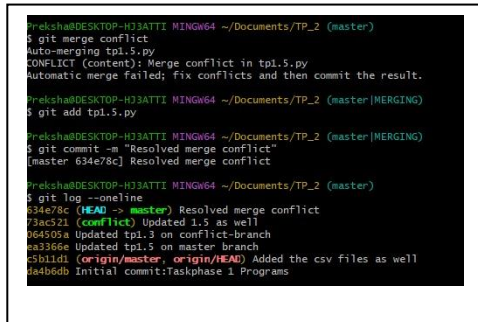
tp1.5.py - C:\Users\Preksha\Documents\TP_2\tp1.5.py (3.14.7)
File Edit Format Run Options Window Help
<<<<<<< HEAD
#this will produce a conflict
=====
#This will prove a conflict
>>>>>>> conflict
hash_table = [[], [], [], [], [], [], [], [], [], []]
n=int(input("Enter number of elements"))
for i in range(n):
    num = int(input("Enter number: "))

```

Fig 3: Merge Conflicts

The conflict was resolved by manually editing tp1.5.py to retain the intended content and removing all three marker lines, after which git add tp1.5.py staged the resolution and git commit -m "Resolved merge conflict" finalized it as merge commit 634e78c.

The resulting git log --oneline output confirmed the complete history — from the initial commit, through the CSV pull, the two divergent branch commits, and finally the resolved merge — with the last remaining step being git push origin master to upload the resolved merge commit back to GitHub.

A terminal window showing the process of resolving a merge conflict in Git. The user is in a directory ~/Documents/TP_2 on the master branch. They attempt to merge a branch named tp1.5, which results in a conflict. The terminal shows the conflict message: "CONFLICT (content): Merge conflict in tp1.5.py". The user then resolves the conflict and commits the result. The terminal shows the commit message: "Resolved merge conflict". Finally, the user runs git log --oneline, which displays the commit history: "634e78c (HEAD -> master) Resolved merge conflict", "73ac521 (conflict) Updated 1.5 as well", "064505a Updated tp1.3 on conflict-branch", "ea3366a Updated tp1.5 on master branch", "3b11db (origin/master, origin/HEAD) Added the csv files as well", and "4e4b6db Initial commit:Taskphase 1 Program".

```
Preksha@DESKTOP-H33ATTI MINGW64 ~/Documents/TP_2 (master)
$ git merge conflict
Auto-merging tp1.5.py
CONFLICT (content): Merge conflict in tp1.5.py
Automatic merge failed; fix conflicts and then commit the result.

Preksha@DESKTOP-H33ATTI MINGW64 ~/Documents/TP_2 (master|MERGING)
$ git add tp1.5.py

Preksha@DESKTOP-H33ATTI MINGW64 ~/Documents/TP_2 (master|MERGING)
$ git commit -m "Resolved merge conflict"
[master 634e78c] Resolved merge conflict

Preksha@DESKTOP-H33ATTI MINGW64 ~/Documents/TP_2 (master)
$ git log --oneline
634e78c (HEAD -> master) Resolved merge conflict
73ac521 (conflict) Updated 1.5 as well
064505a Updated tp1.3 on conflict-branch
ea3366a Updated tp1.5 on master branch
3b11db (origin/master, origin/HEAD) Added the csv files as well
4e4b6db Initial commit:Taskphase 1 Program
```

Fig 4: Merge conflict resolved and all commits

X. CONCLUSION

This task gave me a much deeper, hands-on understanding of Git than I had going in — not just the commands, but why they exist and what problems they actually solve. Working through learngitbranching.js.org helped me visualize concepts like branching, merging, and rebasing that are hard to grasp from documentation alone, since seeing the commit tree move and split made the underlying logic click in a way reading about it never did.

Along the way, practical errors turned out to be the best teachers: initializing my repo in the wrong folder taught me to verify the repo root before staging files, a failed push revealed that branch names like main and master aren't interchangeable defaults, and an initial failed attempt at creating a merge conflict forced me to properly understand how branches diverge from a shared commit.

Overall, this exercise moved me from knowing Git commands in isolation to understanding the workflow they belong to — initializing a repo, committing deliberately, branching for parallel work, syncing with a remote, and resolving the conflicts that come from real collaboration.

XI. ACKNOWLEDGMENT

I would like to thank the Driverless Subsystem seniors for providing the guidance and resources that helped me complete the task with a deeper understanding of the version control system Git and the concepts discussed in this report.

XII. REFERENCES

- [1] Learn Git Branching. [Online]. Available: <https://learngitbranching.js.org/>
- [2] Git, “Git Reference” [Online]. Available: <https://git-scm.com/docs>